

论文教学注记：从 Softmax TD 评估到两阶段 Q 控制

1. 一句话抓住新论文

Xie 已经证明 softmax 能用正的匹配核聚合有正有负的 TD error, 从而做固定策略评估。我们的扩展不是 “恢复 TD 的正负号”, 而是:

1. 把状态 token 扩展成状态—动作 token;
2. 第一阶段 softmax 近似 $\max_a Q(s', a)$;
3. 第二阶段沿 Xie 的方式, 把 signed TD value 写回 $Q(s, a)$ 。

与 Liang-Lai 的区别是: 他们更新全局线性参数 w , 必须产生 deltaphi ; 我们直接更新查询 token 上的 $Q(s, a)$, 因此绕开这个动态乘积。

2. 最容易误解的地方: 正权重不等于正输出

softmax attention 的一个标量输出是

$$y = \sum_k a_k v_k, \quad a_k \geq 0, \quad \sum_k a_k = 1.$$

如果 $v=(-2, 1)$ 、 $a=(0.8, 0.2)$, 那么

$$y = 0.8(-2) + 0.2(1) = -1.4.$$

所以权重全正并不会删掉 value 的负号。softmax 只规定 “怎么平均”, 并不规定 “被平均的数据必须为正”。

Xie 的 value projection 先产生

$$\delta_k = r_k + \gamma v(s'_k) - v(s_k),$$

再输出

$$\sum_k K(s_k, s) \delta_k.$$

这里 $K \geq 0$, 但 δ_k 可正可负。因此 policy evaluation 完全可以工作。

凸包结论到底限制什么

凸包结论只说

$$\sum_k a_k v_k \in \text{conv}\{v_k\}.$$

若 value 本身是 signed TD, 凸包里当然可以有负数。真正受限的是旧 V4 那种

$$\hat{g} = \sum_k a_k \phi_k,$$

因为它只平均裸特征, 没有 δ_k carrier。例如 $\phi=(1,2)$, 目标半梯度取 $\delta=(1,-1)$:

$$g = \frac{1}{2}(1 \cdot 1 - 1 \cdot 2) = -\frac{1}{2},$$

但任何凸组合都在 $[1, 2]$, 所以 V_4 不可能得到 $-1/2$ 。这是 “pure feature readout” 的反例, 不是 “所有 softmax attention” 的反例。

3. 为什么第三篇可以做策略改进, 却不一定用 max

在当前策略 π 已评估准确时,

$$\mathbb{E}_{\pi}[\delta \mid s, a] = 0$$

只说明 $Q=Q^{\pi}$ 。策略改进可以在评估之后单独做:

$$\pi'(s) \in \arg \max_a Q^{\pi}(s, a).$$

由 policy improvement theorem,

$$V^{\pi'}(s) \geq V^{\pi}(s).$$

Liang-Lai 还使用 SARSA 和 actor-critic: 策略随着 actor 或行为策略更新而变化, policy improvement 不需要把 max 塞进每一个 TD residual。

因此正确说法是:

- max 不是所有策略改进的必要条件;
- max 是我们选择 Q-learning/Bellman optimality 路线时的必要运算。

4. 两条路线的核心乘法差异

4.1 Liang-Lai 的参数化路线

设

$$Q_w(s, a) = \phi(s, a)^{\top} w.$$

参数更新需要

$$\Delta w = \frac{\alpha}{n} \sum_k \delta_k \phi_k.$$

delta_k 与 phi_k 都随样本变化, 所以每个样本要先做动态乘积, 再求和。线性 attention 中的二阶矩乘法可以自然产生它。

4.2 我们的核化路线

不维护全局 w , 而是在每个状态动作查询 token 上直接保存 $Q(x)$ 。更新是

$$Q^+(x) = Q(x) + \alpha \sum_k K(x_k, x) \delta_k.$$

这里 attention value 只需要保存标量 delta_k, 不需要形成向量乘积 delta_k phi_k。状态动作特征只负责计算匹配核 K 。

你可以把两者记成:

路线	学习对象	难点
Liang-Lai	全局参数 w	生成 deltaphi
Xie/本文	每个 query 的值	生成 target, 并正确匹配 query

5. 第一阶段: softmax 怎么近似 max

对一个下一状态的动作值 $q_1, \dots, q_m$, 定义

$$p_a = \frac{e^{\beta q_a}}{\sum_b e^{\beta q_b}}, \quad M_\beta(q) = \sum_a p_a q_a.$$

beta 越大, 权重越集中在最大动作上。

严格误差界来自 log-sum-exp:

$$\text{LSE}_\beta(q) = \frac{1}{\beta} \log \sum_a e^{\beta q_a} = M_\beta(q) + \frac{H(p)}{\beta}.$$

又因为

$$\max q \leq \text{LSE}_\beta(q), \quad H(p) \leq \log m,$$

所以

$$0 \leq \max q - M_\beta(q) \leq \frac{\log m}{\beta}.$$

代码位置: model.py 的 GroupedSoftmaxMax。它只调用 torch.softmax, 不会偷偷调用 torch.max。exact max 只存在于单独的 OracleMaxKernelQTD 基线。

6. 第二阶段: signed TD 写回

第一阶段得到 M_{β} 后, 形成

$$\delta_k^\beta = r_k + \gamma M_\beta Q(s'_k) - Q(s_k, a_k).$$

然后

$$K_\eta(x_k, x) = \frac{\exp(\eta \langle \psi(x_k), \psi(x) \rangle)}{\sum_j \exp(\eta \langle \psi(x_j), \psi(x) \rangle)},$$

$$Q^+(x) = Q(x) + \alpha \sum_k K_\eta(x_k, x) \delta_k^\beta.$$

代码位置: KernelizedSoftmaxQTD。验证脚本专门输入 (2, -3) 两个 TD value, 得到一个正更新和一个负更新, 直接证明正的 kernel 没有抹掉符号。

7. Bellman 误差递推怎么推

理想单位核更新为

$$Q_{l+1} = (1 - \alpha)Q_l + \alpha T_\beta Q_l.$$

在右侧加减 T^*Q_l :

$$\begin{aligned} Q_{l+1} - Q^* &= (1 - \alpha)(Q_l - Q^*) \\ &\quad + \alpha(T_\beta Q_l - T^*Q_l) \\ &\quad + \alpha(T^*Q_l - T^*Q^*). \end{aligned}$$

三项分别对应: 保留旧误差、softmax-max 偏差、Bellman contraction。取无穷范数:

$$\|Q_{l+1} - Q^*\|_\infty \leq [1 - \alpha(1 - \gamma)]\|Q_l - Q^*\|_\infty + \frac{\alpha\gamma \log |A|}{\beta}.$$

固定 β 时, 长期误差半径至多

$$\frac{\gamma \log |A|}{\beta(1 - \gamma)}.$$

重要细节: 证明只使用 T^* 的压缩性, 没有假设 T_β 自己也是压缩映射。

8. kernel leakage 与有限上下文

若理想 kernel 是单位阵 I , 真实 attention kernel 是 K , 定义

$$\epsilon_K = \max_x \sum_k |K(k, x) - I(k, x)|.$$

若 $|\delta| \leq B$, kernel 造成的一步误差不超过

$$\alpha B \epsilon_K.$$

再把有限样本的 Bellman 误差记为 ϵ_n , 总递推是

$$e_{l+1} \leq \rho e_l + \alpha \frac{\gamma \log |A|}{\beta} + \alpha B \epsilon_K + \alpha \epsilon_n.$$

在当前完整覆盖、确定性期望实验里, 非单位但可逆的 kernel 可能不改变最终 fixed point, 只拖慢收敛。因此论文把 leakage 解释为 “一步/有限步失真”, 不把它说成普遍的最终误差地板。

9. 新实验应该怎么看

主实验 20 个随机 MDP:

- exact Q iteration 与 oracle kernel 达到约 $7.5e-13$ 数值误差;
- internal softmax 从 $\beta=1$ 的 0.2465 降至 $\beta=20$ 的 0.0113;
- 观测 max gap 始终低于 $\log |A| / \beta$;
- pure-convex signed-free ablation 的 Q error 超过 100;
- 动作数从 2 到 8, 20 步误差从 0.0066 增至 0.0190;
- kernel leakage 高时, 20 步误差明显上升。

不要把 oracle 当成需要击败的学习算法。它是 exact max 上界, 用来隔离误差是否真的来自 softmax greedification。

10. 旧结果现在怎么理解

- V2: softmax evaluation memory + 外部 max target + 外部 δ 读出。
- V5: softmax evaluation memory + 外部 δ softmax(δ/τ) ϕ 读出。
- V4: 无 signed carrier 的纯特征凸组合消融。
- R^2 约 0.92: 特定 token 布局和训练协议的诊断, 不是任意 softmax Transformer 的结构常数。

旧实验没有被删除, 但它们不再承担 “层内实现 Q-learning” 的主结论。

11. 答辩时的三句话

1. “Softmax 的正性约束作用在权重, 不作用在 value; Xie 已经把 signed TD error 放进 value, 所以 policy evaluation 不矛盾。”
2. “Liang-Lai 更新参数 w , 需要 $\delta \phi$; 我们直接更新 query 上的 $Q(s, a)$, 因此把难点转成两阶段的 approximate max 与 kernel matching。”

3. “我们的误差来自 $\log|A|/\beta$ 、kernel distortion 和有限上下文，而不是 TD 正负号消失。”